

Scala

Exercise 1 - events detection

Solving the first exercise consists of using the filter transformation presented in the "Transformations" lesson. But before we can filter anything, we need to prepare the data for this operation. Remember, Apache Kafka stores records as simple arrays of bytes, hence processing them involves deserialization.

Apache Spark Structured Streaming jobs perform the deserialization with the `from_*` functions, such as `from_csv`, `from_avro`, or recently added `from_protobuf`. As you can see, it all depends on the input format of the record. In our case it's a JSON event, so we're using the `from_json`:

```
val schemaForFiltering = StructType(Array(
  StructField("eventId", StringType),
  StructField("os", StringType),
  StructField("visitedPage", StringType)
))

val onlyMacOsEvents = inputStream.withColumn(
  "filtering_struct",
  functions.from_json($"value".cast("string"), schemaForFiltering)
)
```

After this data preparation step, it's time to add the filtering logic and the data enrichment with the extra column:

```
val onlyMacOsEvents = inputStream
  .withColumn("filtering_struct",
    functions.from_json($"value".cast("string"), schemaForFiltering))
  .filter("filtering_struct.os = 'macOS'")
  .selectExpr("filtering_struct.*")
  .withColumn("processedAt", functions.current_timestamp())
```

Although the output generation doesn't look challenging at first glance, it hides a pitfall. The job won't be able to write a flat JSON structure if the processing logic doesn't extract all fields from the nested struct (`selectExpr("filtering_struct.*")`).

Exercise 2 - data reprocessing

The key part here is to reorganize the code so that the logic can be shared between the real-time and data reprocessing jobs. The solution consists of adding a shared place that will

expose one or multiple functions with the shared logic. It turns out for our case, it's enough to define a single method that returns the filtering and data projection logic:

```
def processInputData(inputDataFrame: DataFrame): DataFrame = {
  val schemaForFiltering = StructType(Array(
    StructField("os", StringType),
    StructField("visitedPage", StringType)
  ))

  import inputDataFrame.sparkSession.implicits._
  inputDataFrame
    .select(
      $"key".cast("string"),
      functions.from_json($"value".cast("string"),
                          schemaForFiltering).as("visit"),
      $"value".cast("string")
    )
    .filter("visit.os IN ('macOS', 'Windows', 'Linux')")
    .filter("visit.visitedPage != ''")
    .select($"key", $"value")
}
```

You may be thinking, why these filtering and projection together? If I defined them separately, I would need to repeat them in both jobs. Of course, it's a valid solution if you fully master the scope, for example via the integration tests, but it seems more complex to put in place and is more open for mistakes than a single function. For example, what if you add one filter condition to the real-time code and forget to add it to the reprocessing logic? With the single-method solution you apply the extra logic without asking any other questions.

Another refactored piece is the sink with the SinkConfiguration class. It's a lightweight structure that only holds the output declaration. But having it here avoids redeclaring in the real-time and reprocessing jobs. Again, I could choose an alternative approach and copy the configuration to each job explicitly with the output topic name declared as a constant. Why didn't I implement it? I'm a bit lazy and didn't want to type the full sink configuration for both parts ;)

Exercise 3 - Dead-Letter Queue

To implement the Dead-Letter Queue pattern in Apache Spark Structured Streaming, I'm relying on one of the Apache Kafka sink properties. When you write a record to the broker, you can define the topic in 2 manners:

- either as an `.option("topic", "my_topic")` and in that case, all rows will go to the same place
- or as a topic column (e.g. `.withColumn("topic", get_topic_name())`)

As you can deduce, the topic column allows the destination to be dynamic and come, for example, from a User-Defined Function or the map transformation. I'm implementing here the latter solution:

```
val validAndInvalidVisits = inputStream
  .select(functions.from_json($"value".cast("string"),
    UserVisit.Schema).as("value"))
  .selectExpr("value.*", "key")
  .as[UserVisit]
  .map(visit => {
    // Whoops, the visitedPage is an `Option[String]`, so it can
    // be missing!
    // We call it with a direct `.get`, making the code fail
    // Let's fix it by changing the function and catch any error
    try {
      VisitPageMappingResult(s""""{page":
"${visit.visitedPage.get}""", visit.key, "valid_data")
    } catch {
      case e: Exception => {
        VisitPageMappingResult(visit.asJson, visit.key, "dead_letter")
      }
    }
  })

val writeQuery = validAndInvalidVisits.select("value", "key", "topic")
  .writeStream.format("kafka")
  .options(
    Map(
      "kafka.bootstrap.servers" -> "localhost:29092",
      "checkpointLocation" -> CheckpointDir
    )
  ).start()
```

As you can see, the mapping function deals with the missing visitedPage attribute from a try-catch block. I preferred this approach to catch this and any other errors. In your solution you could also consider an isDefined function on top of the visitedPage optional but be aware that it covers only one possible example.

The mapping function wouldn't be enough. Instead, it's the result type that matters the most. The VisitPageMappingResult looks like that:

```
case class VisitPageMappingResult(value: String, key: String, topic: String)
```

As you can see, it contains the topic name the data sink is using later to forward the data depending on the mapping outcome.

Exercise 4 - backpressure

Apache Spark Structured Streaming is a great streaming framework but unfortunately it may suffer from unexpected latency due to data skew. What does it mean? If one of the tasks takes much more data than the others, such as x2 or x3, it will automatically spend more time processing it. While the data gets processed, the next micro-batch waits and cannot trigger because the micro-batch is sequential. Backpressure is one of the solutions for this issue.

The backpressure implementation consists of 2 operations:

- dividing the input dataset into the records to process now and the records to process in the next micro-batch (eventually)
- combining the records from the previous micro-batch with the records of the current micro-batch

To implement these 2 steps, the easiest solution is to write a sink function as a part of the `foreachBatch` sink. The function first loads the backpressured data, if there are any, and combines it with the new events:

```
.foreachBatch((realTimeDataset: Dataset[RowToProcess], batchNumber: Long) => {
  val maybeBackpressuredDataset = if (batchNumber > 0) {
    Some(
      sparkSession.read.schema(rowToProcessSchema)
        .json(s"${BackpressureDir}/${batchNumber - 1}")
        .as[RowToProcess])
    } else {
      None
    }
  val datasetToProcess = maybeBackpressuredDataset
    .map(backpressuredDataset =>
      backpressuredDataset.union(realTimeDataset))
    .getOrElse(realTimeDataset)
```

The operation either takes the new data or combines it with the records stored in the backpressure files. I'm using the local file system but on production systems you will probably rely on object storage with a configured retention policy (aka TTL) to avoid keeping all backpressured records indefinitely.

After that reading part, I'm defining the backpressure logic as a window function that uses the `row_number()` result to classify the record as an extra one or not:

```
val maxRecordsPerJob = 5
val backpressureWindowSpec = Window
  .partitionBy("id")
  .orderBy($"generation_time".asc)
val datasetWithBackpressureFlag = datasetToProcess
```

```
.withColumn("is_backpressured",
  functions.when(
    functions.row_number().over(backpressureWindowSpec) > maxRecordsPerJob, true
  ).otherwise(false))
```

The next step is to write the classified records either to the final sink or to the backpressure sink. It's not complex as operation but it's important to cache the enriched dataset to avoid reprocessing the input data for both scenarios:

```
cachedDatasetWithBackpressureFlag.filter("is_backpressured = true")
  .drop("is_backpressured")
  .write.format("json").save(s"${BackpressureDir}/${batchNumber}")

cachedDatasetWithBackpressureFlag.filter("is_backpressured = false")
  .drop("is_backpressured")
  .show(truncate = false)

cachedDatasetWithBackpressureFlag.unpersist()
()
```

The writing logic remains simple. The tricky point is to remove the `is_backpressured` column because it's an internal detail already provided by the backpressure storage location.

The function finishes with the `unpersist()` method to release the cache space taken by the `cachedDatasetWithBackpressureFlag`. I'm using the non-blocking method to start the next micro-batch as soon as possible. However, if your main concern is not the latency, you can use the blocking call (`unpersist(blocking=true)`). According to the Scaladoc:

```
/**
 * Mark the Dataset as non-persistent, and remove all blocks for it from memory and
 * disk.
 * This will not un-persist any cached data that is built upon this Dataset.
 *
 * @param blocking Whether to block until all blocks are deleted.
 *
 * @group basic
 * @since 1.6.0
 */
def unpersist(blocking: Boolean): this.type = {
  sparkSession.sharedState.cacheManager.uncacheQuery(
    sparkSession, logicalPlan, cascade = false, blocking)
  this
}
```

Also notice the presence of the `()` in the end of the function. It's the syntax representing Unit, required type to be returned by the `foreachBatch` method.

Python

Exercise 1 - events detection

The Python solution is the copy of the Scala's one but with the PySpark syntax. To start, it also uses the `from_json` function to extract the JSON structure from the input record:

```
schema_for_filtering = StructType([
    StructField("eventId", StringType()),
    StructField("os", StringType()),
    StructField("visitedPage", StringType())
])

only_macos_events = input_data_stream.withColumn("filtering_struct",
F.from_json(F.col("value").cast("string"), schema_for_filtering))\
```

Next, it applies the same filtering and data enrichment step:

```
only_macos_events = input_data_stream.withColumn("filtering_struct",
F.from_json(F.col("value").cast("string"), schema_for_filtering))\
    .filter("filtering_struct.os = 'macOS'")\
    .selectExpr("filtering_struct.*")\
    .withColumn("processedAt", functions.current_timestamp())
```

Finally, the job also extracts the attributes from the struct column (`selectExpr("filtering_struct.*")`) to write them as a flat JSON in the sink.

Exercise 2 - data reprocessing

Remember, the key part here is to avoid redundancy. I achieved that for the Scala solution by extracting the data transformation and the sink configuration to a dedicated structures. The same applies here for Python, starting by the processing logic:

```
from pyspark.sql import DataFrame, functions as F
from pyspark.sql.types import StructType, StructField, StringType

user_visit_schema = StructType([
    StructField("os", StringType()),
    StructField("visitedPage", StringType())
])

def process_input_data(input_data_frame: DataFrame) -> DataFrame:
    return input_data_frame.select(
        F.col("key").cast("string"),
        F.from_json(F.col("value").cast("string"), user_visit_schema).alias("visit"),
        F.col("value").cast("string")
    )
```

```

).filter("visit.os IN ('macOS', 'Windows', 'Linux')")\
  .filter("visit.visitedPage != ''") \
.select(
  F.col("key"), F.col("value")
)

```

The code is relatively easy to understand. The single new thing is the usage of aliased functions `import at` from `pyspark.sql` import functions as `F`. It's not required but nice practice if you want to avoid full functions import or prefixing them with `functions.*` everytime that makes the code:

- harder to read if for the explicit function import because we lose the context of the function's origin
- even more harder to read if we need to precede each function with the module name

Finally, I'm also using the same code sharing technique for the sink parameters. This time I'm exposing them from a function:

```

def get_sink_configuration() -> Dict[str, str]:
    return {
        'kafka.bootstrap.servers': 'localhost:29092',
        'topic': 'valid_visits'
    }

```

The single tricky part here is the requirement for `.options()` in PySpark. The function expects named parameters, such as `.options(numRows=3, truncate=False)` if we wanted to write 3 non truncated rows to the console sink. Fortunately, Python can convert dictionaries into those named parameters with `**` which gives sinks declarations like:

```

# real-time
write_query = records_to_write.writeStream.format("kafka")\
  .option("checkpointLocation",
    "/tmp/bde/module5/homework/e2/checkpoint")\
  .options(**get_sink_configuration())\
  .start()

# reprocessing
records_to_write.write.format("kafka") \
  .options(**get_sink_configuration()) \
  .save()

```

Exercise 3 - Dead-Letter Queue

The Dead-Letter Queue implementation in PySpark relies on the same principle as for the Scala Spark, hence the dynamic topic resolution from the `DataFrame` column. The difference is the column's creation. PySpark `DataFrame` API doesn't have the `map` function and we cannot use the class-based transformation as for the Scala-based solution. However, it's possible to write a User-Defined Function (UDF) with exactly the same code:

```

@pandas_udf(MapType(StringType(), StringType()))
def get_visited_page(input: pandas.Series) -> pandas.Series:
    values_to_clean_as_dict = input.to_dict()
    values_to_return = {}
    for index, json_to_clean in values_to_clean_as_dict.items():
        json_dict = json.loads(json_to_clean)
        try:
            values_to_return[index] = {
                'value': json.dumps({'page': json_dict["visitedPage"]}),
                'topic': 'valid_data'
            }
        except Exception:
            values_to_return[index] = {
                'value': json_to_clean,
                'topic': 'dead_letter'
            }
    return pandas.Series(values_to_return)

```

An important thing to notice here is the usage of the Pandas UDF that under-the-hood is a vectorized function. As a result, instead of executing once for each row, it runs for a bunch of rows every time. It optimizes the execution time by reducing the JVM-to-Python serialization overhead.

The final transformation looks then like that:

```

valid_and_invalid_visits = input_data_stream.withColumn("payload",
                                                         get_visited_page(F.col("value")))

write_query = valid_and_invalid_visits.select("payload.value",
                                              "payload.topic", "key"
                                              )\
    .writeStream.format("kafka")\
    .option("checkpointLocation", "/tmp/bde/module5/homework/e3/checkpoints")\
    .option("kafka.bootstrap.servers", "localhost:29092")\
    .start()

```

Exercise 4 - backpressure

PySpark's backpressure implementation also consists of 2 operations:

- dividing the input dataset into the records to process now and the records to process in the next micro-batch (eventually)
- combining the records from the previous micro-batch with the records of the current micro-batch

To implement these 2 steps, the easiest solution is to write a sink function as a part of the `foreachBatch` sink. The function first loads the backpressured data, if there are any, and combines it with the new events:


```
def print_data_with_backpressure_mechanism(data_frame: DataFrame, batch_version:
int) -> None:
    backpressure_dataset = None
    if batch_version > 0:
        backpressure_dataset =
spark.read.schema(json_schema).json(f'{backpressure_dir}/{batch_version - 1}')

    dataset_to_process = data_frame if not backpressure_dataset else
backpressure_dataset.union(data_frame)
```

As for Scala, the operation either takes the new data or combines it with the records stored in the backpressure files. I'm using the local file system but on production systems you will probably rely on object storage with a configured retention policy (aka TTL) to avoid keeping all backpressured records indefinitely.

After that reading part, I'm defining the backpressure logic as a window function that uses the `row_number()` result to classify the record as an extra one or not:

```
# Classify the records for the backpressure storage
backpressure_window_spec =
Window.partitionBy("id").orderBy(F.asc("generation_time"))
dataset_to_process_with_backpressure_flag = dataset_to_process \
    .withColumn("is_backpressured", F.when(
        F.row_number().over(backpressure_window_spec) > max_records_per_job,
        True).otherwise(False))
```

The next step is to write the classified records either to the final sink or to the backpressure sink. It's not complex as operation but it's important to cache the enriched dataset to avoid reprocessing the input data for both scenarios:

```
cached_dataset_with_backpressure_flag =
dataset_to_process_with_backpressure_flag.cache()

cached_dataset_with_backpressure_flag.filter("is_backpressured =
true").drop("is_backpressured")
.write.format("json") \
    .mode('overwrite').save(f'{backpressure_dir}/{batch_version}')

cached_dataset_with_backpressure_flag.filter("is_backpressured =
false").drop("is_backpressured")
.show(truncate=False)

cached_dataset_with_backpressure_flag.unpersist()
return None
```

The writing logic remains simple. The tricky point is to remove the `is_backpressured` column because it's an internal detail already provided by the backpressure storage location.

The function finishes with the `unpersist()` method to release the cache space taken by the `cached_dataset_with_backpressure_flag`. As for Scala, I'm using the non-blocking method to start the next micro-batch as soon as possible.

In the end, the function returns `None` which is expected by the `foreachBatch`.

Bartosz.